Final Presentation: Star-LoRA for Dataset-Aware Adapter Fine-Tuning

Jihun Park

Waseda University, Fundamental Science and Engineering

2026 年 6 月 28 日

Background: Parameter-Efficient Fine-Tuning (PEFT)

- Large language models achieve strong transfer performance, but full fine-tuning updates all model parameters.
- Low-Rank Adaptation (LoRA) reduces trainable parameters by inserting low-rank update matrices:

$$W' = W + \Delta W, \quad \Delta W = BA, \quad A \in \mathbb{R}^{r \times d}, \ B \in \mathbb{R}^{k \times r}.$$

- The rank r controls the adaptation capacity and computational cost.
- In low-resource settings, a fixed rank can be too large for simple or noisy datasets and too small for complex datasets.

Problem Setting

Research Question

Can AdaLoRA be improved by selecting adapter capacity from dataset characteristics before fine-tuning?

- Standard LoRA: uses a fixed rank and fixed target modules.
- Standard AdaLoRA: adaptively reallocates rank during training, but the initial configuration is usually hand-designed.
- Proposed method: Star-LoRA estimates dataset difficulty before training and uses it to configure AdaLoRA.
- Final task in this project: fake review detection with ModernBERT-large.

Motivation

- Low-resource fine-tuning is sensitive to overfitting and unstable gradient signals.
- Different datasets have different properties:
  - sample count,
  - label imbalance,
  - token sparsity,
  - token-frequency skew,
  - label entropy,
  - input-embedding variance.
- These statistics can indicate whether the model needs smaller, larger, or broader adapter capacity.

Related Works: LoRA

- LoRA freezes the pretrained model and trains low-rank matrices in selected layers.
- It significantly reduces trainable parameters while preserving strong downstream performance.
- A key hyperparameter is the rank r, which is often selected manually.

Limitation

Fixed-rank LoRA does not adapt its capacity to the dataset or to training dynamics.

Related Works: AdaLoRA

- AdaLoRA allocates parameter budget adaptively across LoRA modules.
- It starts from a larger initial rank and prunes/reallocates rank according to importance.
- It improves parameter efficiency compared with fixed-rank LoRA.

Limitation

The base target rank, initial rank, dropout, schedule, and target module set are still usually chosen without explicit dataset profiling.

Gap in Existing Methods

Method	Rank Adaptation	Dataset Profiling	Stability Logging
Full fine-tuning	–	–	–
LoRA	No	No	Optional
AdaLoRA	Yes	No	Optional
Proposed method	Yes	Yes	Yes

- This work focuses on the missing link between dataset statistics and AdaLoRA configuration.

Proposed Method Overview

Dataset-Aware and Stability-Aware AdaLoRA

The method extends AdaLoRA with a pre-training dataset analyzer and a training-time stability monitor.

1. Analyze the training subset before fine-tuning.
2. Convert dataset statistics into AdaLoRA hyperparameters.
3. Select target modules according to dataset complexity.

Dataset Statistics

Statistic	Purpose
N	Low-resource sample size
Class imbalance	Detect label skew and overfitting risk
Token sparsity	Detect limited lexical coverage
Token-frequency skew	Detect vocabulary concentration
Label entropy	Estimate task uncertainty/complexity
Embedding variance	Estimate input diversity in representation space

Dataset-Aware Rank Policy

- The target rank is computed from three scaling factors:

$$r_{\text{target}} = \text{clip}\left(r_{\text{base}} \cdot f_{\text{size}}(N) \cdot g_{\text{dist}} \cdot g_{\text{comp}}, r_{\text{minimize}}, r_{\text{maximize}}\right).$$

- Size factor:

$$f_{\text{size}}(N) = \text{clip}\left(\frac{\log(1+N)}{\log(1+5000)}, 0.3, 1.5\right).$$

- Distribution factor decreases rank when imbalance, sparsity, or skew is high.
- Complexity factor increases rank when entropy or embedding variance is high.

Distribution and Complexity Scaling

$$g_{\text{dist}} = \text{clip}\left(1 - 0.5\left(0.5I_{\text{class}} + 0.3S_{\text{token}} + 0.2K_{\text{token}}\right), 0.5, 1.0\right),$$
$$g_{\text{comp}} = \text{clip}\left(1 + 0.5H_{\text{label}} + 0.3V_{\text{emb}}, 1.0, 2.0\right).$$

- I_{class}: class imbalance.
- S_{token}: token sparsity.
- K_{token}: normalized token-frequency skew.
- H_{label}, V_{emb}: task complexity indicators.

Adaptive AdaLoRA Configuration

- After computing r_{target}, the method sets:

$$r_{\text{init}} = \text{maximize}(r_{\text{target}} + 8,\ 2r_{\text{target}}),$$

$$\alpha = \text{maximize}(4r_{\text{target}},\ 2r_{\text{base}}).$$

- Dropout is increased for very small, imbalanced, or sparse datasets:

$$p_{\text{drop}} \in [0.05, 0.20].$$

- AdaLoRA schedule:

$$t_{\text{init}} = 0.2T, \quad t_{\text{final}} = 0.85T, \quad \Delta t = 0.05T.$$

ModernBERT Target Modules

- ModernBERT does not use the usual `q_proj`/`v_proj` names.
- The implementation expands compact module names to exact layer paths:

$$\{\texttt{model.layers.*.attn.Wqkv},\ \texttt{model.layers.*.attn.Wo}\}.$$

- This keeps LoRA, AdaLoRA, and Star-LoRA comparable because all adapter methods target the same attention modules.
- The difference is the capacity policy: fixed rank for LoRA/AdaLoRA, dataset-aware rank for Star-LoRA.

Final Experimental Setup

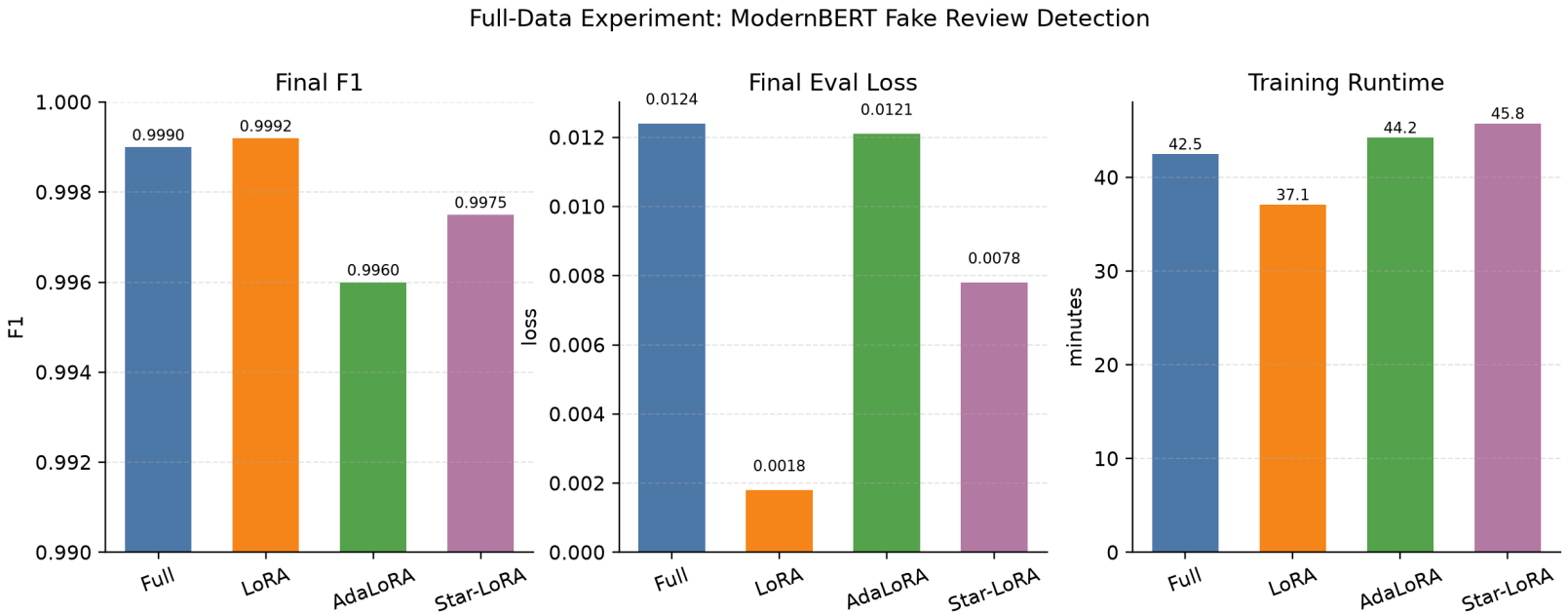
Item	Setting
Task	Binary fake review detection
Dataset	`testing/data/deepseek_synthetic_reviews.jsonl`
Model	`answerdotai/ModernBERT-large`
Train/eval split	90/10, seed 42
Training	5 epochs, batch size 8, grad. accumulation 4
Methods	Full, LoRA, AdaLoRA, Star-LoRA
Metrics	F1, eval loss, runtime, throughput

Baselines and Star-LoRA Configuration

Method	Rank	Init rank	Alpha	Dropout
LoRA	8	–	16	0.10
AdaLoRA	8	12	16	0.10
Star-LoRA	12	24	48	0.05

- Star-LoRA selected a larger rank from the full training set statistics.
- Recorded dataset statistics: balanced labels, token sparsity 0.450, label entropy ≈ 1.0.
- All adapter methods target ModernBERT `Wqkv` and `Wo` modules.

Final Full-Data Results

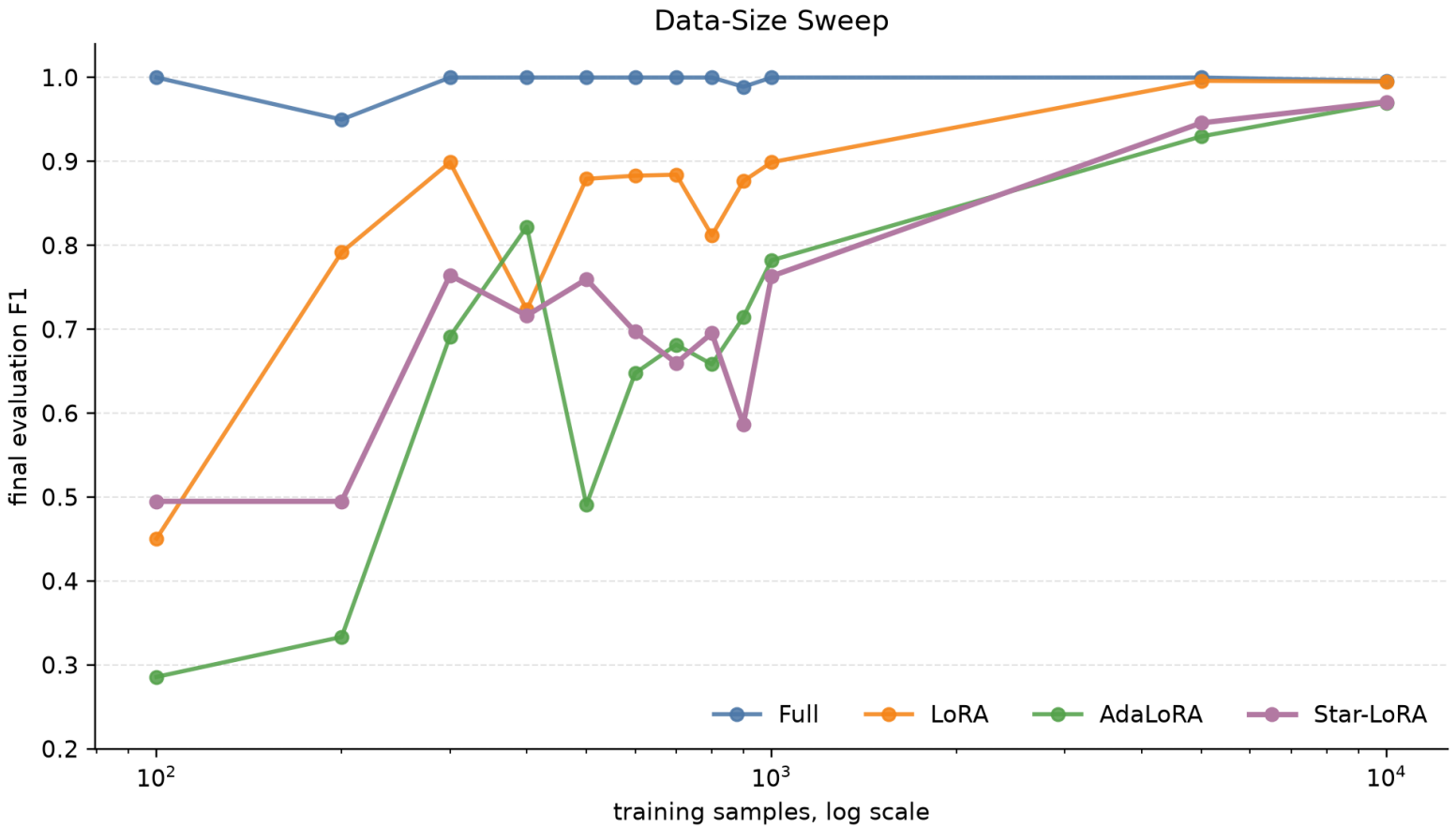


Final Metrics Table

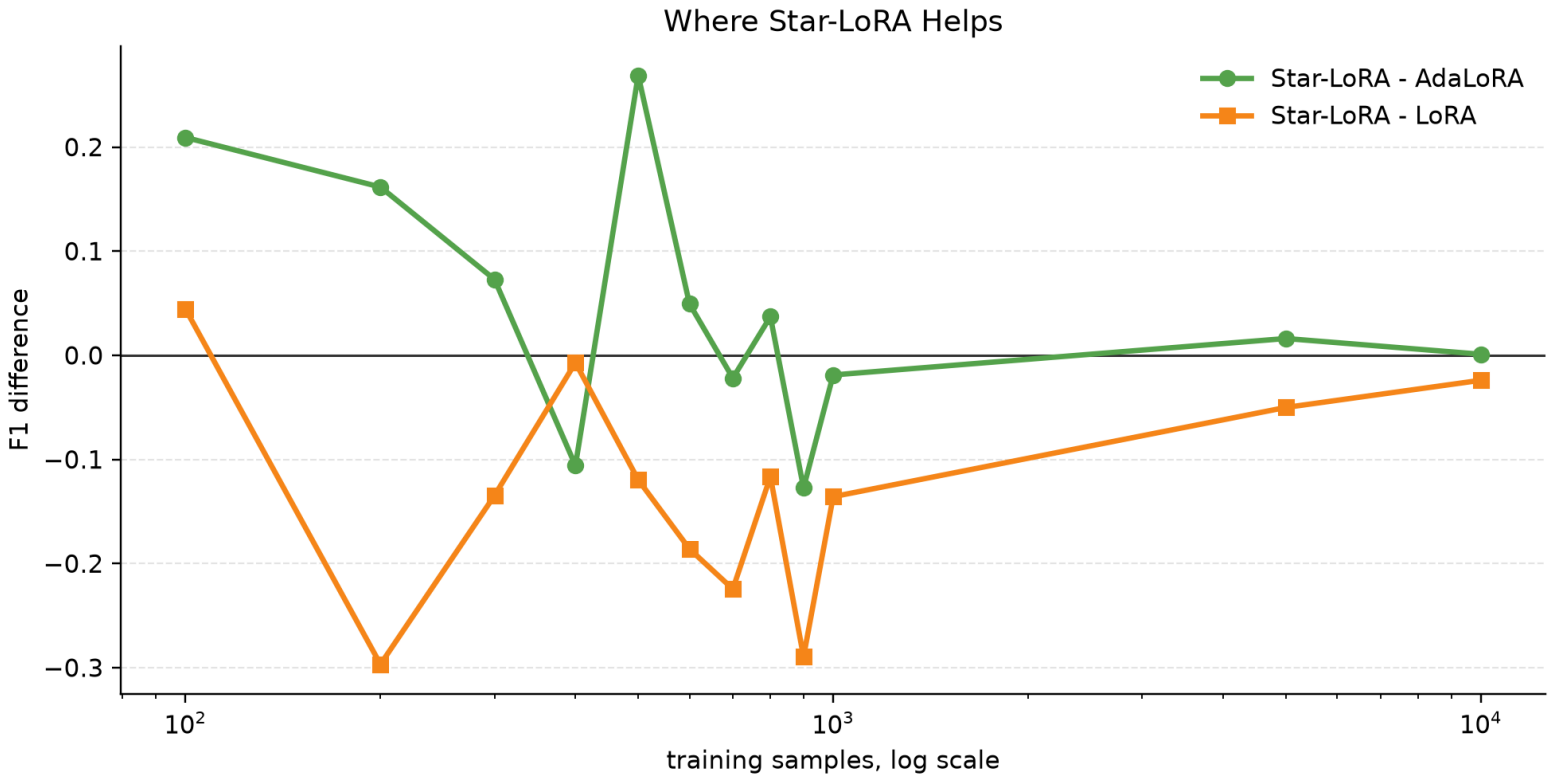
Method	Final F1	Eval loss	Runtime (s)	Samples/s
Full	0.9990	0.0124	2548.62	70.63
LoRA	**0.9992**	**0.0018**	**2225.13**	**80.89**
AdaLoRA	0.9960	0.0121	2654.11	67.82
Star-LoRA	0.9975	0.0078	2745.00	65.57

- Star-LoRA improves over AdaLoRA on the final full-data run.
- Fixed LoRA remains the strongest result on this dataset.

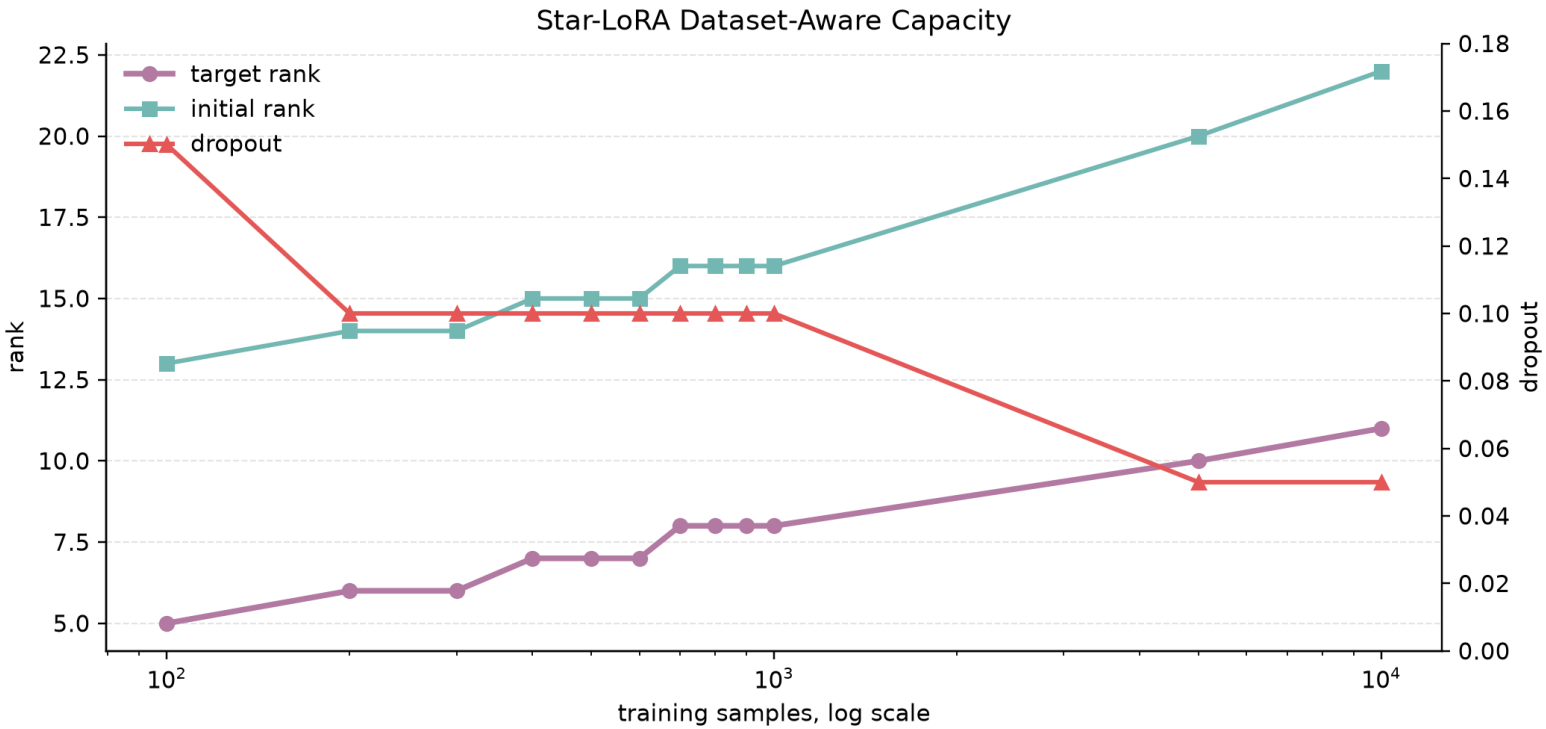
Data-Size Sweep



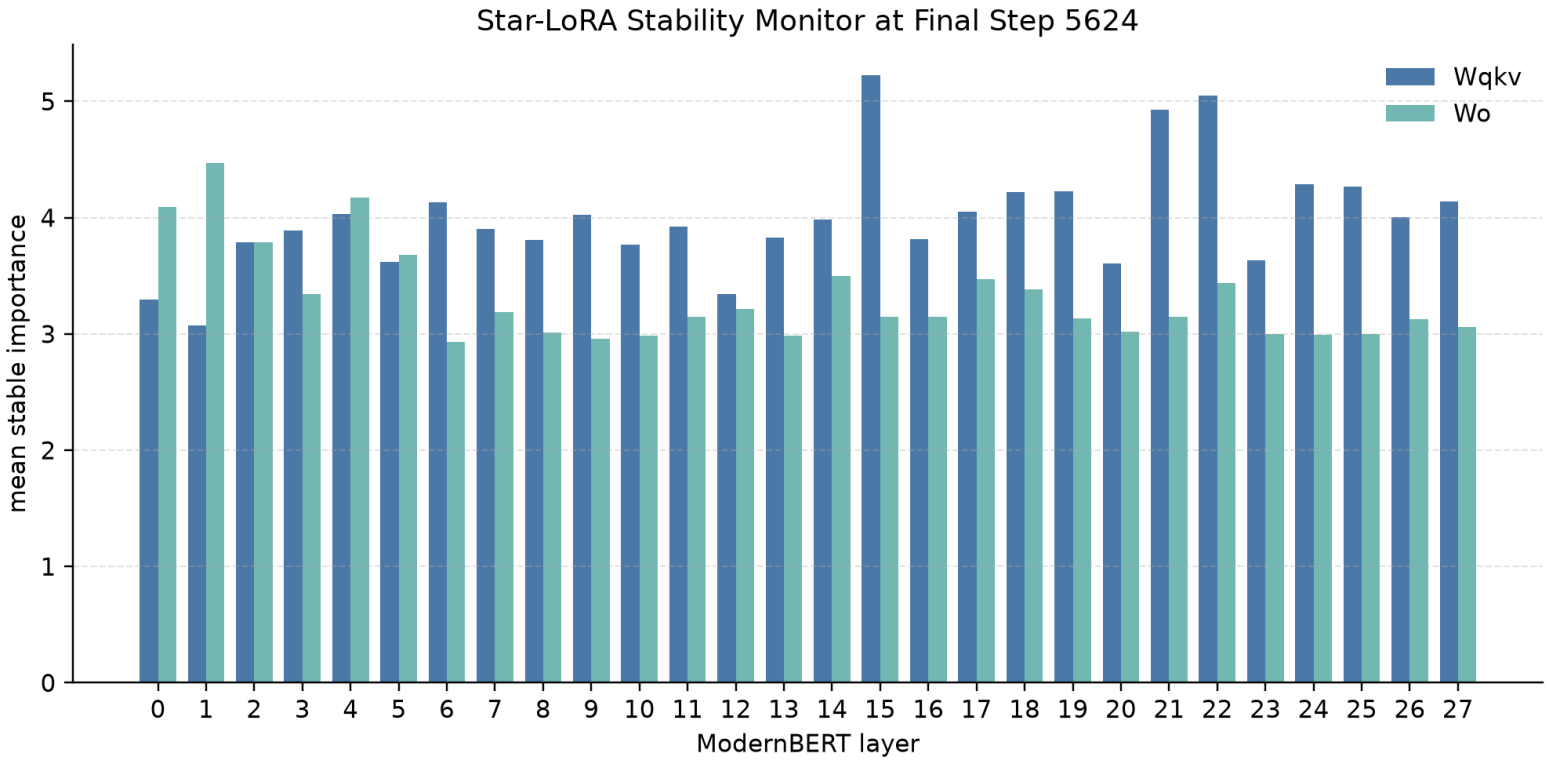
Where Star-LoRA Helps



Dataset-Aware Capacity Selected by Star-LoRA



Stability-Aware Importance Monitor



- The stability log makes layer-wise adapter importance inspectable after training.
- In the final Star-LoRA run, attention input projection `Wqkv` has higher mean stable importance than `Wo`.

Conclusion

Main Finding

Star-LoRA makes AdaLoRA more data-aware and improves over the manually configured AdaLoRA baseline, but fixed LoRA is still the best method on this fake-review dataset.

- Full-data F1: Star-LoRA 0.9975, AdaLoRA 0.9960, LoRA 0.9992.
- In the data-size sweep, Star-LoRA is often better than AdaLoRA at very small and very large sample budgets.
- The rank policy responds to data size by increasing target rank from 5 to 11 in the sweep, and to 12 for the full run.

Limitations

- The current rank policy is hand-designed, so it can over-allocate capacity when a simple fixed-rank LoRA is already sufficient.
- Results are from one dataset split and one major task family: fake review classification.
- Stability scores are logged and visualized, but they are not yet used directly to update PEFT AdaLoRA's internal allocator.
- Star-LoRA has higher runtime than LoRA in the final run because it starts with larger adaptive-rank capacity.

Future Work

- Learn the rank policy from multiple datasets instead of using fixed scaling constants.
- Add an ablation study for each statistic: size, sparsity, skew, entropy, and embedding variance.
- Use stability scores during training, not only as post-hoc logging.
- Test whether Star-LoRA helps more on noisier, imbalanced, or lower-resource review datasets.